Digital Image Processing

Assignment 2 – Detailed Solutions

Chapter 3: Intensity Transformations & Spatial Filtering

Contents

1 Problem 3.4: Bit Plane Slicing Transformations **3**
- 1.1 Understanding Bit Planes (Section 3.2.4) . . . 3
- 1.2 Binary Representation Example . . . 3
- 1.3 Why Bit Planes Matter . . . 3
- 1.4 The Solution: Intensity Slicing Transformation . . . 4
- 1.5 Intensity Ranges for Each Bit Plane . . . 4
- 1.6 Transformation Graphs for All 8 Bit Planes . . . 4
- 1.7 Complete Dry Run: Extract All Bits from r = 167 . . . 5
- 1.8 Complete Example: 3×3 Image Bit Plane Extraction . . . 5

2 Problem 3.5: Effect of Zeroing Bit Planes on Histogram **7**
- 2.1 Part (a): Zeroing Lower-Order Bits . . . 7
- 2.2 Part (b): Zeroing Higher-Order Bits . . . 8

3 Problem 3.6: Why Histogram Equalization Isn't Perfectly Flat **9**
- 3.1 The Theory (Section 3.3.1) . . . 9
- 3.2 Why It Fails in Discrete Images . . . 9

4 Problem 3.7: Second Pass Produces Same Result **11**
- 4.1 Understanding the Question . . . 11
- 4.2 Recap: Histogram Equalization Formula . . . 11
- 4.3 Step 1: Result of First Equalization . . . 11
- 4.4 Step 2: Apply Equalization Again (Second Pass) . . . 12
- 4.5 Visual Interpretation: CDF Comparison . . . 12
- 4.6 Why Does This Happen? . . . 13
- 4.7 Practical Implication . . . 13

5 Problem 3.13: Histogram of Combined Images **14**
- 5.1 Understanding the Problem . . . 14
- 5.2 The Core Problem: What Histograms Tell Us . . . 14
- 5.3 Concrete Counterexample . . . 14
- 5.4 Computing A + B (Opposite Arrangement) . . . 15
- 5.5 Computing A + C (Same Arrangement) . . . 15
- 5.6 Analysis of All Operations . . . 16
- 5.7 The Answer and Condition . . . 16
- 5.8 Special Cases Where It IS Possible . . . 16

6 Problem 3.15: Efficient Box Filter Algorithm **17**
6.1 Understanding Box Filter (Section 3.5.1) . . . 17
6.2 The Naive Approach . . . 17
6.3 Part (a): The Efficient Sliding Window Algorithm . . . 17
6.4 Complete Algorithm Steps . . . 18
6.5 Part (b): Computational Advantage Analysis . . . 18

7 Problem 3.19: Computing Median Efficiently **20**
7.1 Understanding Median Filter (Section 3.5.2) . . . 20
7.2 Part (a): Basic Median Computation Procedure . . . 20
7.3 Why Sorting is Expensive . . . 21
7.4 Part (b): Efficient Histogram-Based Update (Huang's Algorithm) . . . 21
7.4.1 Algorithm Steps . . . 21
7.5 Finding Median from Histogram . . . 22
7.6 Computational Advantage . . . 22

8 Problem 3.24: Proving Laplacian is Isotropic **23**
8.1 Understanding Isotropy (Section 3.6.2) . . . 23
8.2 Visual Intuition . . . 23
8.3 Setting Up the Coordinate Rotation . . . 23
8.4 Computing Partial Derivatives Under Rotation . . . 24
8.5 The Key Step: Adding the Second Derivatives . . . 24
8.6 Discrete Laplacian Kernels . . . 25

9 Problem 3.28: Laplacian ≈ Unsharp Masking **26**
9.1 Understanding Unsharp Masking (Section 3.6.3) . . . 26
9.2 Unsharp Masking Process Flow . . . 26
9.3 Understanding Laplacian Sharpening . . . 27
9.4 Connecting the Two Methods . . . 27
9.4.1 Discrete Laplacian Approximation . . . 27
9.4.2 Relating to Local Average . . . 27
9.5 Final Proof: Equivalence . . . 28
9.6 Visual Comparison . . . 28
9.7 Practical Implications . . . 28

1 Problem 3.4: Bit Plane Slicing Transformations

Question

Problem 3.4: A common implementation of intensity slicing is to highlight one range of intensities $[A, B]$ by mapping it to white (255) and mapping all other intensities to black (0). Propose a set of intensity-slicing transformations capable of producing all the individual bit planes of an 8-bit monochrome image.

1.1 Understanding Bit Planes (Section 3.2.4)

What is a Bit Plane?

An 8-bit image has pixel values 0-255. Each value can be written as 8 binary digits. A **bit plane** extracts just ONE bit position from every pixel, creating a binary image.
Pixel decomposition:

$$\text{pixel value} = \sum_{k=0}^{7} b_k \cdot 2^k = b_7 \cdot 128 + b_6 \cdot 64 + b_5 \cdot 32 + \cdots + b_0 \cdot 1$$

where $b_k \in \{0, 1\}$ is the k-th bit.

1.2 Binary Representation Example

Pixel value 167:

$$167 = 128 + 32 + 4 + 2 + 1 = 1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

MSB ← → LSB

	128	64	32	16	8	4	2	1
167 =	**1**	**0**	**1**	**0**	**0**	**1**	**1**	**1**
	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0

1.3 Why Bit Planes Matter

Bit 7	**Bit 6**	**Bit 5**	...	**Bit 0**
50% info	25% info	12.5% info		<1% info

→ Decreasing significance →

- **Higher bit planes (7,6,5):** Contain image structure, edges, main features
- **Lower bit planes (2,1,0):** Contain fine details, often appear noise-like
- **Applications:** Image compression (discard LSBs), watermarking (hide data in LSBs)

1.4 The Solution: Intensity Slicing Transformation

Key Point

To extract bit plane k, use intensity slicing that maps to 255 where bit k is 1:
Transformation function:

$$T_k(r) = \begin{cases} 255 & \text{if } \left\lfloor \frac{r}{2^k} \right\rfloor \mod 2 = 1 \\ 0 & \text{if } \left\lfloor \frac{r}{2^k} \right\rfloor \mod 2 = 0 \end{cases}$$

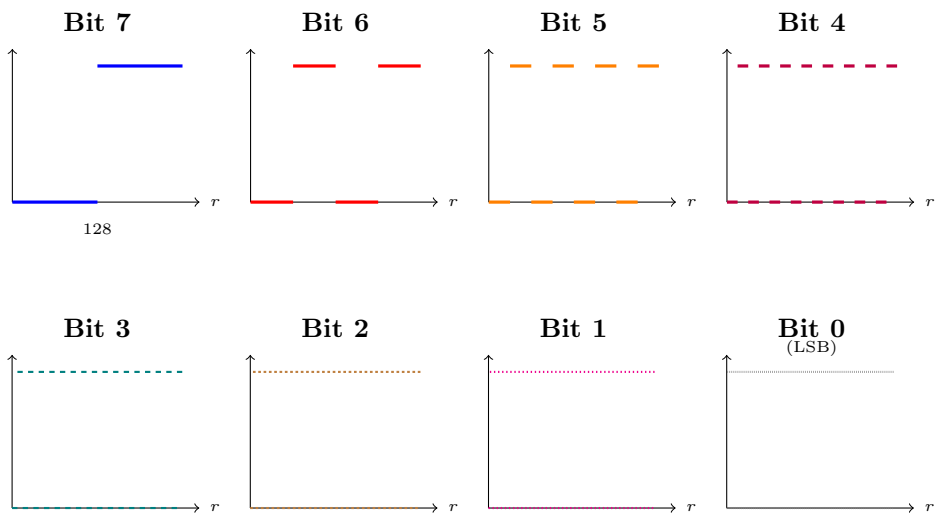
How it works:

1. $\lfloor r/2^k \rfloor$ = shift binary right by k positions
2. $\mod 2$ = extract the rightmost bit (which was bit k)

1.5 Intensity Ranges for Each Bit Plane

Bit	Weight	Ranges mapped to 255 (white)
7 (MSB)	128	[128, 255]
6	64	[64, 127], [192, 255]
5	32	[32, 63], [96, 127], [160, 191], [224, 255]
4	16	8 intervals of width 16 each
0 (LSB)	1	All odd numbers: 1, 3, 5, 7, ..., 255

1.6 Transformation Graphs for All 8 Bit Planes



Pattern: Bit plane k alternates every 2^k intensity values!

1.7 Complete Dry Run: Extract All Bits from r = 167

Bit k	2^k	$\lfloor 167/2^k \rfloor$	**Binary**	**mod 2**	**Output**
7	128	1	00000001	**1**	255 (white)
6	64	2	00000010	**0**	0 (black)
5	32	5	00000101	**1**	255 (white)
4	16	10	00001010	**0**	0 (black)
3	8	20	00010100	**0**	0 (black)
2	4	41	00101001	**1**	255 (white)
1	2	83	01010011	**1**	255 (white)
0	1	167	10100111	**1**	255 (white)

Verification: Reading the "mod 2" column: 1-0-1-0-0-1-1-1 = binary of 167 ✓

1.8 Complete Example: 3×3 Image Bit Plane Extraction

Example

Given this 3×3 image, extract Bit Planes 7 and 2:

Step 1: Original Image with Decimal Values

Original Image

45	130	200
100	167	50
220	75	180

Step 2: Convert to Binary

Binary Representation

00101101	10000010	11001000
01100100	10100111	00110010
11011100	01001011	10110100

Step 3: Extract Bit Plane 7 (check if value ≥ 128)

Bit Plane 7 (MSB)

0	1	1
0	1	0
1	0	1

Bit Plane 2

1	0	0
1	1	0
1	0	1

Observation: Bit 7 shows major bright/dark regions. Bit 2 shows finer patterns.

2 Problem 3.5: Effect of Zeroing Bit Planes on Histogram

Question

Problem 3.5: Discuss the effect that setting to zero the lower-order bit planes would have on the histogram of an image in general. What would be the effect on the histogram if we set to zero the higher-order bit planes instead?

2.1 Part (a): Zeroing Lower-Order Bits

Setting Lower Bits to Zero

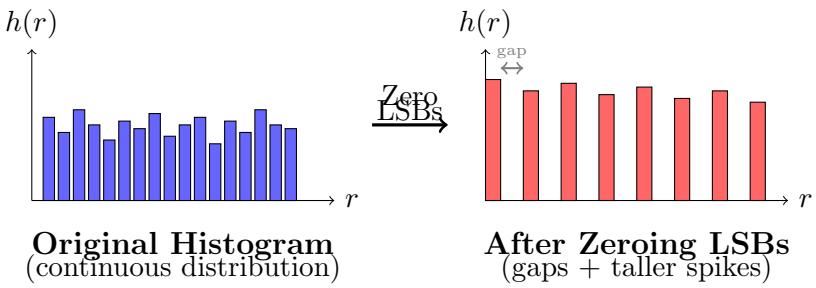
Zeroing the k least significant bits forces all values to be **multiples of** 2^k:

$$\text{new value} = \left\lfloor \frac{\text{old value}}{2^k} \right\rfloor \times 2^k$$

Visual Example: Zero the 4 lowest bits ($2^4 = 16$):

Before		**After**
$167 = 10100111$	Zero 4 LSBs $\longrightarrow$	$10100000 = 160$
$173 = 10101101$		$10100000 = 160$
$162 = 10100010$		$10100000 = 160$

All three different values collapse to 160!
Effect on Histogram:



Key Point

Effects of zeroing k lower bits:

1. **Gaps appear:** Only $256/2^k$ possible values remain
2. **Spikes form:** Multiple values merge into fewer, taller bars
3. **Visual effect:** Posterization/false contouring (visible bands)

2.2 Part (b): Zeroing Higher-Order Bits

Setting Higher Bits to Zero

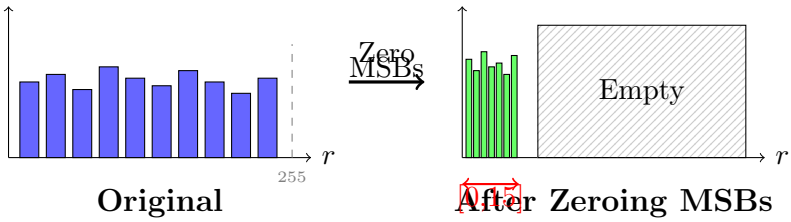
Zeroing the k most significant bits confines all values to range $[0, 2^{8-k} - 1]$:

$$\text{new value} = \text{old value} \mod 2^{8-k}$$

Example: Zero the 4 highest bits:

- $167 = 10100111 \rightarrow 00000111 = 7$
- $200 = 11001000 \rightarrow 00001000 = 8$
- $255 = 11111111 \rightarrow 00001111 = 15$

All values now in range [0, 15]!



Key Point

Effects of zeroing k higher bits:

1. **Compression:** All values squeezed into $[0, 2^{8-k} - 1]$
2. **Image darkens:** Loses dynamic range
3. **Visual effect:** Nearly black image

3 Problem 3.6: Why Histogram Equalization Isn't Perfectly Flat

Question

Problem 3.6: Explain in detail why histogram equalization generally will not yield a flat histogram for images whose pixels have a limited number of distinct gray levels.

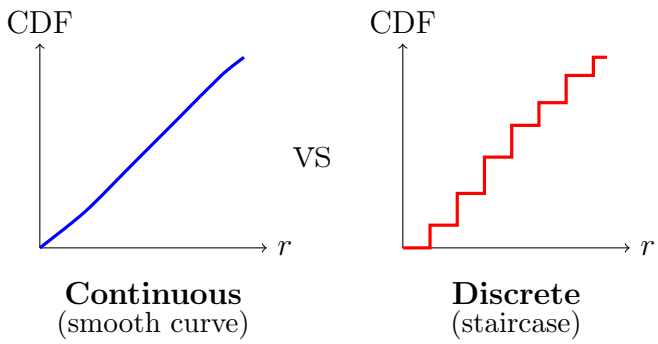
3.1 The Theory (Section 3.3.1)

The transformation:

$$s = T(r) = (L-1)\int_0^r p_r(w)\,dw = (L-1)\cdot \mathrm{CDF}(r)$$

In **continuous** mathematics, this guarantees a uniform output.

3.2 Why It Fails in Discrete Images



Three Fundamental Problems

1. **Rounding:** CDF values must be rounded to integers.
2. **Many-to-One:** Different inputs can map to same output (spikes).
3. **Empty Bins:** Some outputs receive no pixels (gaps).

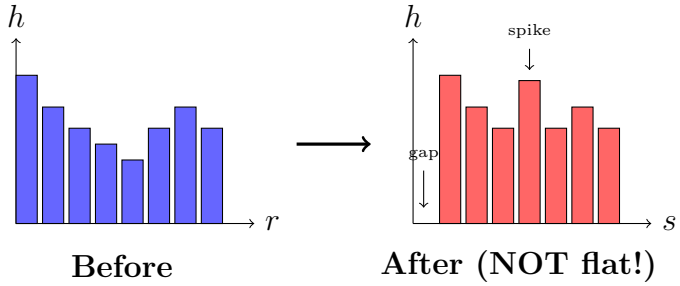
Example

3-bit image (8 levels: 0-7) with 4200 pixels:

r_k	Count	CDF	$s_k = 7 \times \mathrm{CDF}$	Output
0	800	0.190	1.33	$\rightarrow$ **1**
1	600	0.333	2.33	$\rightarrow$ **2**
2	500	0.452	3.17	$\rightarrow$ **3**
3	400	0.548	3.83	$\rightarrow$ **4**
4	300	0.619	4.33	$\rightarrow$ **4**
5	500	0.738	5.17	$\rightarrow$ **5**
6	600	0.881	6.17	$\rightarrow$ **6**
7	500	1.000	7.00	$\rightarrow$ **7**

Problems:

- $s = 0$ gets **no pixels** (gap!)
- $s = 4$ gets pixels from $r = 3$ AND $r = 4$ (spike!)



4 Problem 3.7: Second Pass Produces Same Result

Question

Problem 3.7: Prove that a second pass of histogram equalization will yield exactly the same result as the first pass.

4.1 Understanding the Question

What We Need to Prove

If we apply histogram equalization to an image:

1. **First pass:** Original image $f \to$ Equalized image g
2. **Second pass:** Equalized image $g \to$ Result h

Claim: $h = g$ (the second pass doesn't change anything!)

4.2 Recap: Histogram Equalization Formula

The transformation function for histogram equalization is:

$$s = T(r) = (L-1)\int_0^r p_r(w)\,dw = (L-1)\cdot \mathrm{CDF}(r)$$

where:

- r = input intensity (0 to $L-1$)
- s = output intensity
- $p_r(r)$ = probability density function (normalized histogram) of input
- CDF = cumulative distribution function
- L = number of gray levels (typically 256)

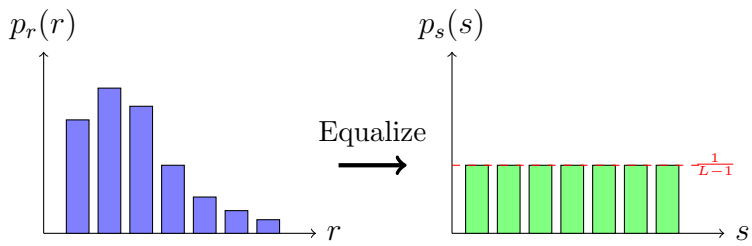
4.3 Step 1: Result of First Equalization

After First Pass

After histogram equalization, the output s has a **uniform distribution**:

$$p_s(s) = \frac{1}{L-1} \quad \text{for } 0 \le s \le L-1$$

This is the **goal** of histogram equalization — to spread intensities evenly across the entire range.



Before: Non-uniform **After:** Uniform (flat)

4.4 Step 2: Apply Equalization Again (Second Pass)

Now we apply histogram equalization to the already-equalized image s:

$$z = T(s) = (L-1)\int_0^s p_s(w)\,dw$$

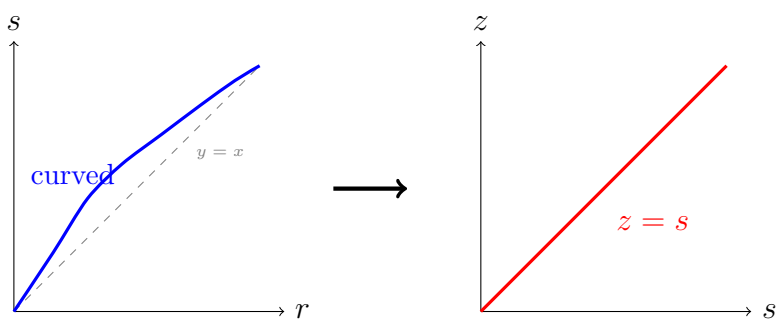
Since $p_s(s) = \dfrac{1}{L-1}$ (uniform):

$$\begin{aligned} z &= (L-1)\int_0^s \frac{1}{L-1}\,dw \\ &= (L-1)\cdot\frac{1}{L-1}\int_0^s dw \\ &= (L-1)\cdot\frac{1}{L-1}\cdot s \\ &= s \end{aligned}$$

Key Point

Result: $z = s$
The second equalization produces the **identity transformation** — output equals input!

4.5 Visual Interpretation: CDF Comparison



First Pass
CDF of original image

Second Pass
CDF of uniform = straight line (Identity!)

4.6 Why Does This Happen?

The Mathematical Reason

The CDF of a uniform distribution is a **straight line**:

$$\text{CDF}(s) = \int_0^s \frac{1}{L-1} dw = \frac{s}{L-1}$$

When we multiply by $(L-1)$:

$$T(s) = (L-1) \cdot \frac{s}{L-1} = s$$

A straight-line CDF produces the **identity transformation**!

4.7 Practical Implication

Key Point

Conclusion:

1. Histogram equalization is **idempotent**: applying it multiple times gives the same result as applying it once
2. Once an image is equalized, it stays equalized — no further "improvement" is possible
3. This is because a uniform distribution is already the "optimal" distribution that equalization tries to achieve

In practice: If you accidentally run histogram equalization twice on an image, don't worry — the result is the same as running it once!

5 Problem 3.13: Histogram of Combined Images

Question

Problem 3.13: Consider two images $f(x, y)$ and $g(x, y)$, and let h_f and h_g denote the histograms of these two images. Give the conditions under which you can compute the histograms of: (a) $f + g$, (b) $f - g$, (c) $f \times g$, (d) $f \div g$ from h_f and h_g only.

5.1 Understanding the Problem

Point-Wise Operations

When we write $f + g$, we mean **pixel-by-pixel** addition:

$$(f + g)(x, y) = f(x, y) + g(x, y) \quad \text{for each pixel location } (x, y)$$

Same applies for $f - g$, $f \times g$, and $f \div g$.
Key insight: The result at each pixel depends on **which specific values** are at that exact location in BOTH images.

5.2 The Core Problem: What Histograms Tell Us

Histogram Limitations

A histogram tells us:

- How many pixels have each intensity value
- The overall distribution of intensities

A histogram does NOT tell us:

- **WHERE** those pixels are located in the image
- Which pixel at position (x, y) in f pairs with which value at (x, y) in g
- Any spatial information whatsoever

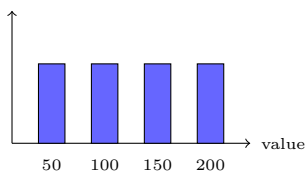
Image 1

50	100	150
200	50	100
150	200	50

Image 2

100	150	50
50	200	150
200	100	50

SAME Histogram!



3 pixels each

5.3 Concrete Counterexample

Consider three 2×2 images with **identical histograms**:

Image A

~~10~~	~~20~~
~~30~~	~~40~~

Image B

~~40~~	~~30~~
~~20~~	~~10~~

Image C

~~10~~	~~20~~
~~30~~	~~40~~

Critical observation: $h_A = h_B = h_C = \{10:1,\ 20:1,\ 30:1,\ 40:1\}$

5.4 Computing A + B (Opposite Arrangement)

$$\underbrace{\begin{array}{|c|c|}\hline 10 & 20 \\ \hline 30 & 40 \\ \hline\end{array}}_{\textbf{A}} + \underbrace{\begin{array}{|c|c|}\hline 40 & 30 \\ \hline 20 & 10 \\ \hline\end{array}}_{\textbf{B}} = \underbrace{\begin{array}{|c|c|}\hline \mathbf{50} & \mathbf{50} \\ \hline \mathbf{50} & \mathbf{50} \\ \hline\end{array}}_{\textbf{A+B}}$$

Pixel-by-pixel calculation:

Position	**A value**	**B value**	**A+B**
(0,0)	10	40	50
(0,1)	20	30	50
(1,0)	30	20	50
(1,1)	40	10	50

$h_{A+B} = \{50:4\}$ — All pixels have the SAME value!

5.5 Computing A + C (Same Arrangement)

$$\underbrace{\begin{array}{|c|c|}\hline 10 & 20 \\ \hline 30 & 40 \\ \hline\end{array}}_{\textbf{A}} + \underbrace{\begin{array}{|c|c|}\hline 10 & 20 \\ \hline 30 & 40 \\ \hline\end{array}}_{\textbf{C}} = \underbrace{\begin{array}{|c|c|}\hline \mathbf{20} & \mathbf{40} \\ \hline \mathbf{60} & \mathbf{80} \\ \hline\end{array}}_{\textbf{A+C}}$$

Pixel-by-pixel calculation:

Position	**A value**	**C value**	**A+C**
(0,0)	10	10	20
(0,1)	20	20	40
(1,0)	30	30	60
(1,1)	40	40	80

$h_{A+C} = \{20:1,\ 40:1,\ 60:1,\ 80:1\}$ — Four DIFFERENT values!

5.6 Analysis of All Operations

Operation	A op B	A op C
$+$ (addition)	All 50's: $h = \{50 : 4\}$	All different: $h = \{20, 40, 60, 80\}$
$-$ (subtraction)	All 0's (or ± 30)	All 0's
$\times$ (multiply)	$\{400, 600, 600, 400\}$	$\{100, 400, 900, 1600\}$
$\div$ (divide)	$\{0.25, 0.67, 1.5, 4\}$	All 1's

Same histograms ($h_B = h_C$) produce **different results**!

5.7 The Answer and Condition

Key Point

Answer for ALL operations $(+, -, \times, \div)$**:**
You **CANNOT** compute the result histogram from h_f and h_g alone under general conditions.
Proof by counterexample:

- $h_B = h_C$ (identical individual histograms)
- But $h_{A+B} \neq h_{A+C}$ (completely different result histograms)

What information is needed instead:
The **joint histogram** $h(f, g)$ which counts how many pixels have $f(x, y) = a$ AND $g(x, y) = b$ simultaneously.

5.8 Special Cases Where It IS Possible

When Can We Compute Result Histogram?

Only under very restrictive conditions:
1. Constant image: If one image is constant ($g(x, y) = c$ for all pixels)

- Then h_{f+g} is just h_f shifted by c

2. Independent images: If the locations of intensity values in f and g are statistically independent (random), we can compute expected histogram using probability.
3. Same image: If $g = f$, then $h_{f+f} = h_{2f}$ can be computed.
In general practical cases, **none of these hold**.

6 Problem 3.15: Efficient Box Filter Algorithm

Question

Problem 3.15: (a) Develop a procedure for computing the box-filter as the filter moves from point to point. (b) Determine the computational advantage over straightforward implementation.

6.1 Understanding Box Filter (Section 3.5.1)

What is a Box Filter?

A **box filter** (also called *averaging filter* or *mean filter*) replaces each pixel with the **average** of all pixels in its $n \times n$ neighborhood.
Mathematical Definition:

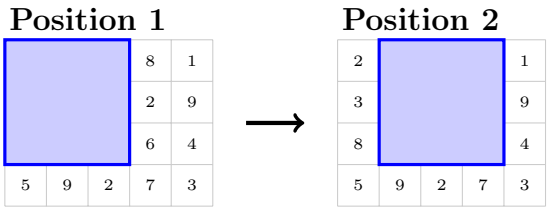
$$g(x, y) = \frac{1}{n^2} \sum_{i=-k}^{k} \sum_{j=-k}^{k} f(x + i, y + j) \quad \text{where } n = 2k + 1$$

Applications:

- Noise reduction (smoothing)
- Blurring for privacy/artistic effects
- Preprocessing for edge detection

6.2 The Naive Approach

For each output pixel, we must sum ALL n^2 values in the window:



Sum all 9 values
$= 2 + 4 + 6 + 3 + 7 + 5 + 8 + 1 + 3 = 39$

Sum all 9 AGAIN
$= 4 + 6 + 8 + 7 + 5 + 2 + 1 + 3 + 6 = 42$

Problem: For each output pixel, we recompute everything from scratch!

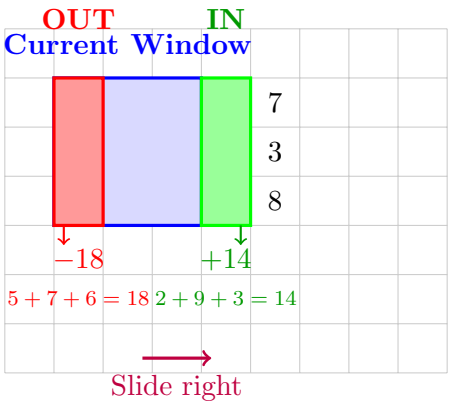
6.3 Part (a): The Efficient Sliding Window Algorithm

Key Insight

When the window slides one pixel right, **only ONE column changes**:

- One column **leaves** on the left $\rightarrow$ **subtract** its sum
- One column **enters** on the right $\rightarrow$ **add** its sum

The other $(n - 1)$ columns stay the same — no need to recompute!



$$S_{new} = S_{old} - (\text{left column}) + (\text{right column}) = 36 - 18 + 14 = 32$$

6.4 Complete Algorithm Steps

1. **Initialize:** Compute full sum for first window position: $S_0 = \sum_{i,j} f(i,j)$
2. **Precompute Column Sums:** Store the n-pixel sum of each column in the row

$$C_j = \sum_{i=0}^{n-1} f(x+i, j) \quad \text{for each column } j$$

3. **Slide Horizontally:** For each step right:

$$S_{x+1} = S_x - C_{left} + C_{right}$$

   Average $= S/(n^2)$

4. **Move to Next Row:** Update column sums (subtract top row, add bottom row):

$$C_j^{new} = C_j^{old} - f(top, j) + f(bottom, j)$$

Example

Detailed Walkthrough for 3×3 Box Filter:
Given image region and 3×3 filter:

10	20	30	40	50
15	25	35	45	55
12	22	32	42	52

Step 1: First window

$C_0 = 10 + 15 + 12 = 37$
$C_1 = 20 + 25 + 22 = 67$
$C_2 = 30 + 35 + 32 = 97$
$S_1 = 37 + 67 + 97 = 201$
Avg $= 201/9 = 22.33$

10	20	30	40	50
15	25	35	45	55
12	22	32	42	52

Step 2: Slide right

$C_3 = 40 + 45 + 42 = 127$
$S_2 = S_1 - C_0 + C_3$
$S_2 = 201 - 37 + 127 = 291$
Avg $= 291/9 = 32.33$

6.5 Part (b): Computational Advantage Analysis

Operation Count Comparison

For an $n \times n$ filter on an $M \times N$ image:
Naive Method:

- Operations per pixel: n^2 additions
- Total: $M \times N \times n^2$ operations

Efficient Sliding Method:

- Horizontal slide: 2 operations (subtract left + add right)
- Update column sums per row: n columns $\times$ 2 ops $= 2n$ operations total
- Average per pixel: $\approx 2n$ operations

Method	**Ops/pixel**	$n = 5$	$n = 21$	$n = 101$
Naive	n^2	25	441	10,201
Efficient	$2n$	10	42	202
Speedup	$\frac{n^2}{2n} = \frac{n}{2}$	**2.5×**	**10.5×**	**50.5×**

Key Point

Key Results:

1. Speedup factor $= \frac{n}{2}$ (larger filters benefit more!)
2. Time complexity: $O(n^2) \to O(n)$ per pixel
3. For large filters (e.g., 101×101), get **50× speedup**
4. Algorithm is **independent of filter size** for horizontal traversal — each step is just 2 operations!

7 Problem 3.19: Computing Median Efficiently

Question

Problem 3.19: (a) Give a procedure for computing the median of an $n \times n$ neighborhood. (b) Propose a technique for updating the median as the center moves from pixel to pixel.

7.1 Understanding Median Filter (Section 3.5.2)

What is Median Filtering?

The **median filter** replaces each pixel with the **median** (middle value when sorted) of all pixels in its neighborhood.
Why use median instead of mean?

- **Removes salt-and-pepper noise** — extreme outliers don't affect the median
- **Preserves edges** — unlike mean filter which blurs edges
- The median is a **robust statistic** — less sensitive to outliers

7.2 Part (a): Basic Median Computation Procedure

1. **Extract:** Collect all n^2 pixel values from the $n \times n$ neighborhood
2. **Sort:** Arrange values in ascending (or descending) order
3. **Select:** Median = value at position $\dfrac{n^2+1}{2}$

Example

Complete Walkthrough for 3×3 Neighborhood:

Step 1: Extract

23	45	12
67	89	34
56	78	41

Center pixel = 89

Step 2: Sort

Unsorted: {23, 45, 12, 67, 89, 34, 56, 78, 41}
collect ↓
Sorted: {12, 23, 34, 41, **45**, 56, 67, 78, 89}

Step 3: Select Median

$n^2 = 9$ values, median at position $\dfrac{9+1}{2} = 5$

MEDIAN ↓

pos 1	pos 2	pos 3	pos 4	pos 5	pos 6	pos 7	pos 8	pos 9
12	23	34	41	45	56	67	78	89

Result: Center pixel 89 → 45

7.3 Why Sorting is Expensive

Computational Cost of Basic Approach

For each pixel:

- Extract n^2 values: $O(n^2)$
- Sort using comparison sort: $O(n^2 \log n^2) = O(n^2 \log n)$
- Select median: $O(1)$

Total per pixel: $O(n^2 \log n)$
For a 1000×1000 image with 21×21 filter:

$$1000 \times 1000 \times 441 \times \log(441) \approx 3.8 \times 10^{12} \text{ comparisons!}$$

7.4 Part (b): Efficient Histogram-Based Update (Huang's Algorithm)

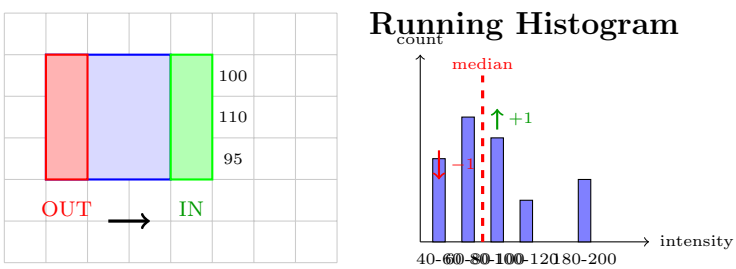
Key Insight

For 8-bit images (values 0-255), maintain a **running histogram** of the window:

- **256 bins** — one for each possible intensity value
- When window slides: **update** histogram (don't rebuild)
- Find median by **scanning** histogram (not sorting)

7.4.1 Algorithm Steps

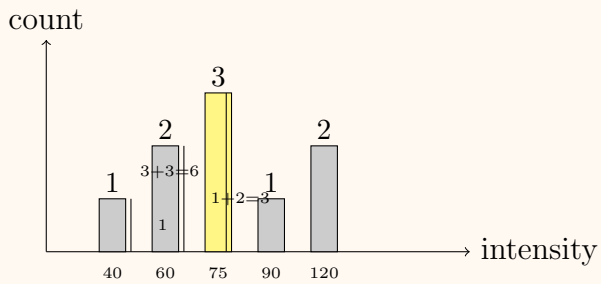
1. **Initialize:** Build histogram $H[0..255]$ for first window
2. **Track median:** Keep count of values $\leq$ current median
3. **When sliding right:**
   - For each value v in **leaving column**: $H[v] \leftarrow H[v] - 1$
   - For each value v in **entering column**: $H[v] \leftarrow H[v] + 1$
4. **Adjust median:**
   - If removed values were $\leq$ old median: decrease count
   - If added values were $\leq$ old median: increase count
   - Scan histogram to find new median position



7.5 Finding Median from Histogram

Example

Given histogram with $n^2 = 9$ pixels, find median (5th value):



Need position 5: Cumulative reaches 5 at intensity **75**
Scan: 1 (at 40) $\rightarrow$ 3 (at 60) $\rightarrow$ **6** (at 75) ≥ 5 ✓

7.6 Computational Advantage

Method	**Operations per Pixel**	**For** $n = 21$
Naive (sort each time)	$O(n^2 \log n)$	$\sim$ 2,900
Histogram-based (Huang)	$O(n + L)$ where $L = 256$	$\sim$ 300
Speedup	—	$\sim$ **10**$\times$

Key Point

Huang's Algorithm Benefits:

1. Window update: $O(n)$ — only process leaving/entering columns
2. Median search: $O(L)$ where L = number of gray levels (256)
3. Total: $O(n + L)$ vs $O(n^2 \log n)$ for naive
4. **Larger filters benefit more** from this optimization

8 Problem 3.24: Proving Laplacian is Isotropic

Question

Problem 3.24: Show that the Laplacian $\nabla^2 f = \dfrac{\partial^2 f}{\partial x^2} + \dfrac{\partial^2 f}{\partial y^2}$ is isotropic (rotation invariant).

8.1 Understanding Isotropy (Section 3.6.2)

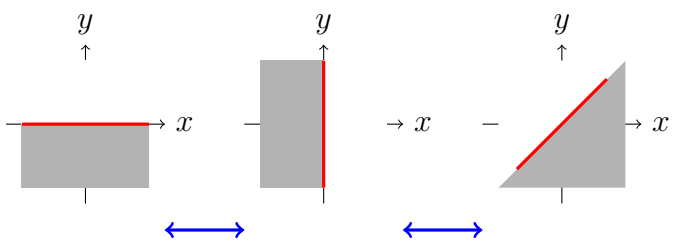
What is Isotropy?

An operator is **isotropic** (or **rotation invariant**) if it produces the **same response** regardless of the edge orientation in the image.
Why it matters for edge detection:

- An edge at 0, 45, 90, or any angle should be detected equally well
- Anisotropic operators have "blind spots" for certain orientations
- Isotropy ensures uniform sensitivity in all directions

8.2 Visual Intuition

Horizontal Edge Vertical Edge Diagonal Edge



$\nabla^2 f = k$ $\nabla^2 f = k$ $\nabla^2 f = k$

Same response for ALL orientations!

8.3 Setting Up the Coordinate Rotation

To prove isotropy, we show that $\nabla^2 f$ is unchanged when we **rotate** the coordinate system by angle θ.

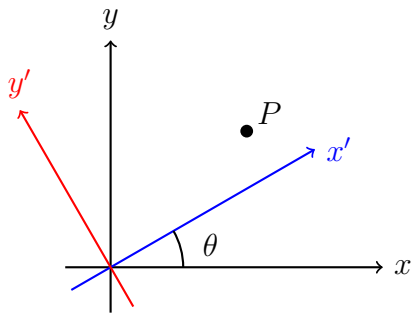
Coordinate Rotation Transformation

Rotate coordinates (x, y) to new coordinates (x', y'):

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x' \\ y' \end{pmatrix}$$

Explicitly:

$$x = x' \cos\theta - y' \sin\theta$$
$$y = x' \sin\theta + y' \cos\theta$$



8.4 Computing Partial Derivatives Under Rotation

Using the chain rule, express derivatives in the new coordinates:

First derivatives:

$$\frac{\partial f}{\partial x'} = \frac{\partial f}{\partial x} \cdot \frac{\partial x}{\partial x'} + \frac{\partial f}{\partial y} \cdot \frac{\partial y}{\partial x'} = f_x \cos\theta + f_y \sin\theta$$

$$\frac{\partial f}{\partial y'} = \frac{\partial f}{\partial x} \cdot \frac{\partial x}{\partial y'} + \frac{\partial f}{\partial y} \cdot \frac{\partial y}{\partial y'} = -f_x \sin\theta + f_y \cos\theta$$

Second derivatives (applying chain rule again):

$$\frac{\partial^2 f}{\partial x'^2} = \frac{\partial^2 f}{\partial x^2} \cos^2\theta + 2\frac{\partial^2 f}{\partial x \partial y} \sin\theta\cos\theta + \frac{\partial^2 f}{\partial y^2} \sin^2\theta$$

$$\frac{\partial^2 f}{\partial y'^2} = \frac{\partial^2 f}{\partial x^2} \sin^2\theta - 2\frac{\partial^2 f}{\partial x \partial y} \sin\theta\cos\theta + \frac{\partial^2 f}{\partial y^2} \cos^2\theta$$

8.5 The Key Step: Adding the Second Derivatives

$$\begin{aligned}
\nabla'^2 f &= \frac{\partial^2 f}{\partial x'^2} + \frac{\partial^2 f}{\partial y'^2} \\
&= \frac{\partial^2 f}{\partial x^2} \underbrace{(\cos^2\theta + \sin^2\theta)}_{=1} + \frac{\partial^2 f}{\partial y^2} \underbrace{(\sin^2\theta + \cos^2\theta)}_{=1} \\
&\quad + 2\frac{\partial^2 f}{\partial x \partial y}(\sin\theta\cos\theta - \sin\theta\cos\theta) \\
&= \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = \nabla^2 f \quad \checkmark
\end{aligned}$$

Key Point

The **magic** happens because:

1. $\sin^2\theta + \cos^2\theta = 1$ (Pythagorean identity)
2. The cross term f_{xy} cancels out: $+2f_{xy}\sin\theta\cos\theta - 2f_{xy}\sin\theta\cos\theta = 0$

Result: $\nabla'^2 f = \nabla^2 f$ for ANY rotation angle θ!

8.6 Discrete Laplacian Kernels

Approximating the Laplacian

The continuous Laplacian is approximated discretely using convolution kernels:

4-neighbor

0	1	0
1	-4	1
0	1	0

Not isotropic

8-neighbor

1	1	1
1	-8	1
1	1	1

More isotropic

The 8-neighbor version better approximates the true (continuous) Laplacian's isotropy.

9 Problem 3.28: Laplacian ≈ Unsharp Masking

Question

Problem 3.28: Show that subtracting the Laplacian from an image is proportional to unsharp masking (Section 3.6.3).

9.1 Understanding Unsharp Masking (Section 3.6.3)

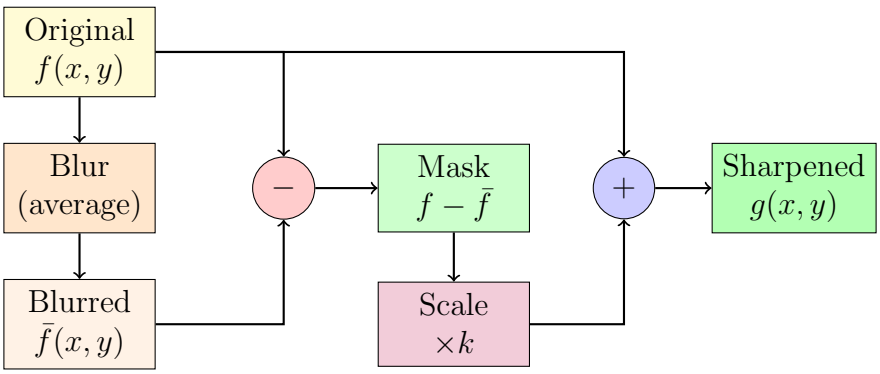
What is Unsharp Masking?

A classic image sharpening technique that works by:

1. Create a **blurred** ("unsharp") version of the image
2. **Subtract** the blur from the original to get the "mask" (edges/details)
3. **Add** the mask back to enhance edges

Name origin: The "unsharp" mask is created by blurring; adding it back makes the image sharper!

9.2 Unsharp Masking Process Flow



Mathematical formulation:

$$\boxed{g(x, y) = f(x, y) + k \cdot \underbrace{[f(x, y) - \bar{f}(x, y)]}_{\text{unsharp mask}}}$$

where:

- f = original image
- $\bar{f}$ = blurred image (local average)
- k = sharpening strength (typically $k > 0$)
- g = sharpened output

9.3 Understanding Laplacian Sharpening

Laplacian as Edge Detector

The Laplacian detects edges by computing the **second derivative**:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Properties:

- **Positive** where intensity increases rapidly (one side of edge)
- **Negative** where intensity decreases rapidly (other side)
- **Zero** in flat regions
- Responds to edges regardless of direction (isotropic)

Laplacian sharpening formula:

$$\boxed{g(x, y) = f(x, y) - c \cdot \nabla^2 f(x, y)}$$

where $c > 0$ is a scaling constant. We **subtract** because the Laplacian gives negative values at edges we want to enhance.

9.4 Connecting the Two Methods

9.4.1 Discrete Laplacian Approximation

The continuous second derivative is approximated using differences:

$$\frac{\partial^2 f}{\partial x^2} \approx f(x+1, y) - 2f(x, y) + f(x-1, y)$$

$$\frac{\partial^2 f}{\partial y^2} \approx f(x, y+1) - 2f(x, y) + f(x, y-1)$$

Adding them:

$$\begin{aligned}\nabla^2 f &\approx f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y) \\ &= \underbrace{[f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)]}_{\text{sum of 4 neighbors}} - 4f(x, y)\end{aligned}$$

9.4.2 Relating to Local Average

The local average of 4 neighbors:

$$\bar{f}(x, y) = \frac{1}{4}[f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)]$$

Therefore:

$$\nabla^2 f = 4\bar{f} - 4f = -4(f - \bar{f})$$

Key Point

The key relationship:

$$\boxed{\nabla^2 f \approx -4(f - \bar{f})}$$

The Laplacian is proportional to the **negative** of the unsharp mask!

9.5 Final Proof: Equivalence

Starting with Laplacian sharpening:

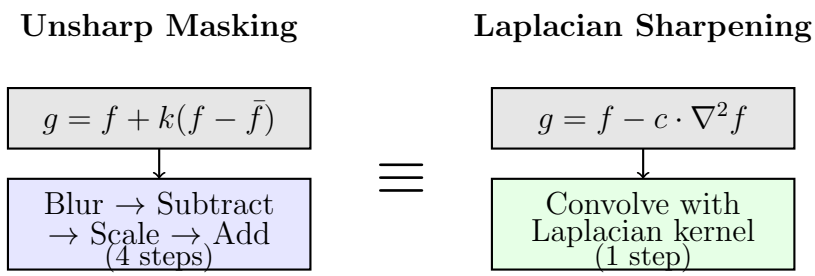
$$\begin{aligned} g &= f - c \cdot \nabla^2 f \\ &= f - c \cdot (-4)(f - \bar{f}) \\ &= f + 4c(f - \bar{f}) \end{aligned}$$

Setting $k = 4c$:

$$\boxed{g = f + k(f - \bar{f})}$$

This is exactly the unsharp masking formula!

9.6 Visual Comparison



9.7 Practical Implications

Key Point

Both methods achieve the same goal:

1. Extract high-frequency detail (edges) from the image
2. Add this detail back to enhance sharpness

Relationship between parameters:

- Unsharp masking: sharpening strength k
- Laplacian: scaling constant $c = k/4$

When to use which:

- **Unsharp masking:** More intuitive parameters, variable blur kernel size
- **Laplacian:** Single convolution operation, computationally simpler

Example

Numerical example showing equivalence:
Given a 3×3 region with center pixel value 120 and its 4 neighbors:

—	100	—
80	**120**	90
—	110	—

Center pixel = **120**

Step-by-step calculations:

1. **Local average** (4 neighbors):

$$\bar{f} = \frac{100 + 80 + 90 + 110}{4} = \frac{380}{4} = 95$$

2. **Unsharp mask** $(f - \bar{f})$:

$$f - \bar{f} = 120 - 95 = 25$$

3. **Discrete Laplacian**:

$$\nabla^2 f = (100 + 80 + 90 + 110) - 4 \times 120 = 380 - 480 = -100$$

4. **Verify the relationship**:

$$-4(f - \bar{f}) = -4 \times 25 = -100 = \nabla^2 f \quad \checkmark$$

Conclusion: The Laplacian equals -4 times the unsharp mask, confirming the equivalence!

End of Solutions